Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 22, 2026
© 2026 Arpit Raj. All rights reserved.

Object-Oriented Programming vs. Arrays & Namespaces

Why OOP over Arrays?

Arrays solve the problem of storing data, OOP organizes the data, related operations and rules together. Also with arrays more feature means more coordination, for new features create new arrays, but every function should coordinate correct arrays and correct indices.

We can also use struct with arrays to achieve the same thing as classes.
i.e.

```
struct Bankaccount {
    string s;
    int b;
}
Bankaccount accounts[1000];
```

In C++, struct and class are very similar, default difference is member access (public for struct, private for class).

> **Note:**
> It's not like Arrays are incapable of doing the work. It's just that classes provide a structured way to group the data with ops related to it, along with access controls and regulations.

Modular Approach

OOP provides a modular approach to group each entity data and operation in its own class. It also helps to keep different types of entities separate.

Eg:

```
class Bank account {
    ...
}
class Bank employee {
    ...
}
```

(Organizes entities and separates different types of entities)

Functional program can also do it using modules via separate functions and arrays.
Eg:

```
// Bank account module
string account names[1000];
int balance[1000];
void deposit()
void withdraw()

// Bank employee module
string employee name[1000];
void showemployee()
```

However the employee module have access to bank account data which is irrelevant and vice versa. Restriction can be made via scope design and namespaces.

© Arpit Raj, LNMIIT Jaipur

Namespaces vs. Encapsulation

Eg:

```
namespace Account Module {
    namespace { // -> internal to this source file
        int balance = 1000;
    }
    void deposit() {}
    void show bal() {}
}

namespace employee module {
    namespace {
        int salary = 8000;
    }
    void incsalary()
    void decsalary()
}

int main() {
    Account Module :: deposit(500)
}
```

However this isn't encapsulation like OOP offers.
Eg:

```
namespace Employee Module {
    namespace {
        string employeeId_name;
        }
        func1() // -> a function which can get name of employee Id
        func2()

}

namespace account module {
    void showrestrictedinfo(int id) {
        string name = Employee Module :: func1(id)
        // now a separate module can get name
    }
}
```

However if func1 was inside namespace too it is also bound to internal linkage and account module cant access it through employee Module :: func1().

- This nested modular approach using namespace is not encapsulation in strict sense.
- It groups related data and functions and hides data from other entities but since there is no class or access control mechanism it cant be classified as OOPS encapsulation.

Core Concept of OOP (OOPS)

- Organizes a program around entities (objects).
- Groups entities (objects) of same type into classes and bundles their data with operations that manages that data.